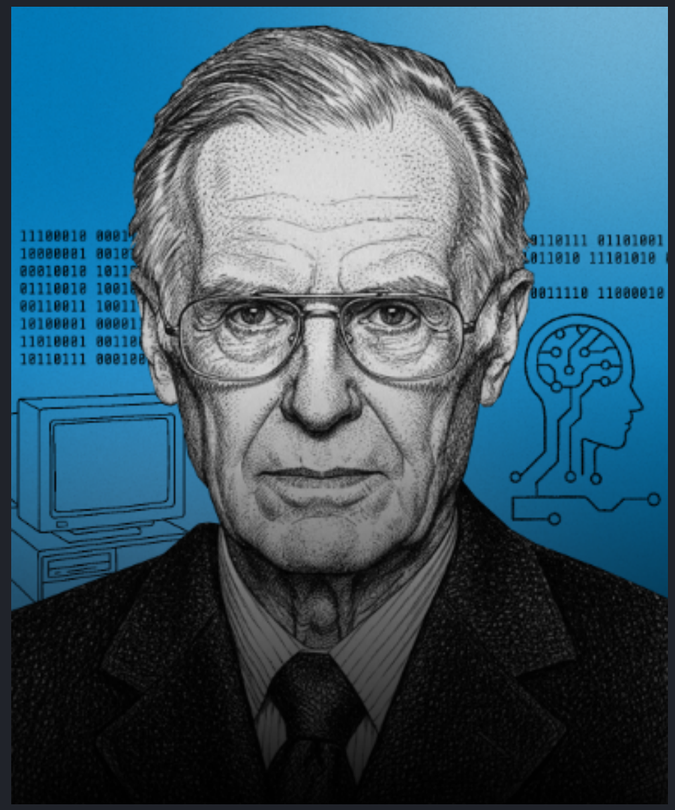
Compiladores e Intérpretes

Apuntes



Amanda Ramírez Viales

05 de noviembre del 2026

Contenidos

01 Propiedades de parsers LL(1)

02 Parsers LL(2)

03 Parsers LL(k)

04 Descenso recursivo

05 Errores y LL(1)

06 Mensajes de error

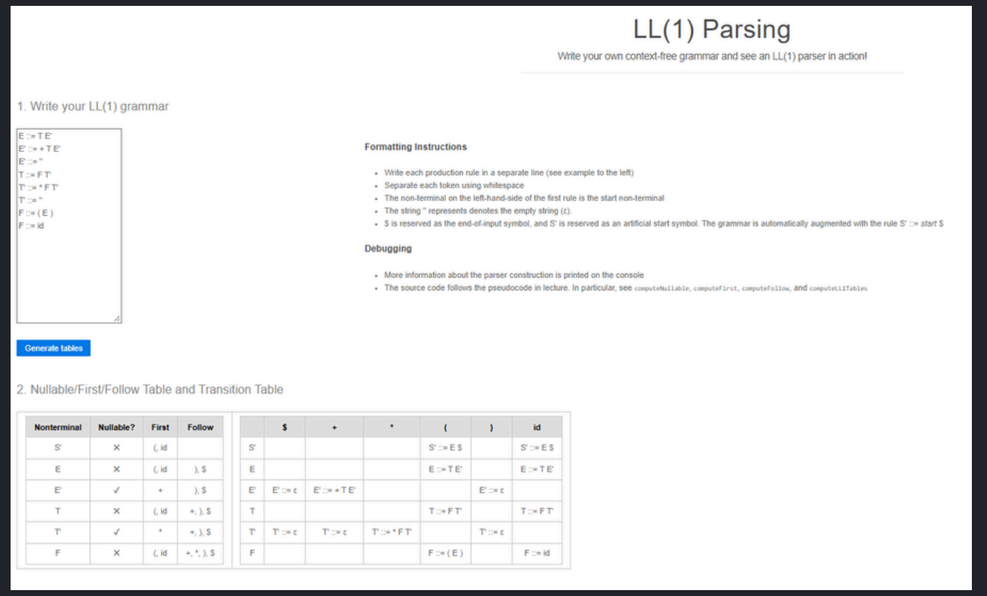
Propiedades Pasers LL(1)

Sea G LL(1)

1. Si una hilera no se rechaza, se obtiene la derivación más izquierda que la generó.
2. G no es ambigua.
3. Tiempo y espacio lineales respecto al largo de la hilera.

Las gramáticas LL(1) son un subconjunto propio de las CFG: muchas no se pueden parsear así.

El parsing LL(1) crece en proporción directa al largo de la entrada.



LL(1) Parsing

Write your own context-free grammar and see an LL(1) parser in action!

1. Write your LL(1) grammar

E ::= T E'
E' ::= + T E'
E' ::= ε
T ::= F T'
T' ::= * F T'
T' ::= ε
F ::= (E)
F ::= id

Formatting Instructions

- Write each production rule in a separate line (see example to the left)
- Separate each token using whitespace
- The non-terminal on the left-hand-side of the first rule is the start non-terminal
- The string ε represents denotes the empty string (ε).
- $ is reserved as the end-of-input symbol, and S' is reserved as an artificial start symbol. The grammar is automatically augmented with the rule S' ::= *start* $

Debugging

- More information about the parser construction is printed on the console
- The source code follows the pseudocode in lecture. In particular, see `computeNullable`, `computeFirst`, `computeFollow`, and `computeLLTables`

Generate tables

2. Nullable/First/Follow Table and Transition Table

Nonterminal	Nullable?	First	Follow
S'	✗	(, id	
E	✗	(, id	), $
E'	✓	+	), $
T	✗	(, id	+,), $
T'	✓	*	+,), $
F	✗	(, id	+, *,), $

	$	+	*	(	)	id
S'				S' ::= E $		S' ::= E $
E				E ::= T E'		E ::= T E'
E'	E' ::= ε	E' ::= + T E'			E' ::= ε	
T				T ::= F T'		T ::= F T'
T'	T' ::= ε	T' ::= ε	T' ::= * F T'		T' ::= ε	
F				F ::= (E)		F ::= id

Parsers LL(2)

En un parser LL(1) se puede predecir la regla que debe usarse con solo un símbolo de lookahead.

Hay gramáticas que no son LL(1), aunque no sean recursivas por la izquierda y estén factorizadas.

Puede ser que viendo 2 símbolos por adelantado se pueda predecir la regla que debe usarse.



+ Se definen las funciones:
    - FIRST2() * Todos los posibles inicios
    - FOLLOW2()
    - PREDICT2()

* Cálculos son mucho más caros.

* La tabla de LL(2) tiene (n^2 + n) columnas, donde n es la cantidad de terminales de la gramática.

Tabla de parsing LL(2)

1. S → (S) S
2. S → ε

	((	()	($	)(	))	)$
S						

PARSERS LL(K)

Caso general: parsers LL(K)

Un lookahead de K símbolos para predecir qué regla usar.

+ Hay
    - FIRST_K()
    - FOLLOW_K()
    - PREDICT_K()

Tablas con (n^K + n^(K-1)) columnas, donde n es la cantidad de terminales de la gramática.

Casi todos los lenguajes de programación son LL(1) o, si no, se puede usar otro tipo de parsers.

¿Recuerdan MICRO?



- Primer proyecto: compilador del lenguaje MICRO.
- Su gramática era LL(1), pero no se usó ninguna tabla.
- Se usó la técnica de descenso recursivo, equivalente a LL(1).
- Si una gramática es LL(1), se puede hacer por descenso recursivo, y viceversa.
- Ambas técnicas tienen los mismos requisitos.
- Es sorprendentemente fácil, pero solo para gramáticas pequeñas.

Descenso recursivo

- Siempre hay un token actual de la hilera.

- Cada no terminal tiene un procedimiento asociado.
- El token actual decide qué regla aplicar.

- El lado de cada regla de la gramática se convierte en llamadas a los procedimientos de cada No Terminal o un MATCH del símbolo actual con el terminal esperado.



Ejemplo 1:

```
E  → i E'
E' → + i E' | ε

main() {
  c = gettoken();
  E();
}
```

```
E() {
  match('i');
  E'();
}

match(char t) {
  if (t == c)
    c = gettoken();
  else
    printf("ERROR");
}
```

```
E'() {
  if (c == '+') {
    match('+');
    match('i');
    E'();
  } else
    return;
}
```

Ejemplo 2:

```
SQ → S SQ'
SQ' → ; SQ | ε
S  → <statement>

<statement> es un token.

main() {
  c = gettoken();
  SQ();
}
```

```
SQ() {
  S();
  SQ'();
}

S() {
  match('<statement>');
}
```

```
SQ'() {
  if (c == ';') {
    match(';');
    SQ();
  } else
    return;
}

match(char t) {
  if (t == c)
    c = gettoken();
  else
    printf("ERROR");
}
```

Ejemplo 3:

```
S   → IFS | <other>
IFS → if ( EX ) S EL
EL  → else S | ε
EX  → 0 | 1

main() {
  c = gettoken();
  S();
}
```

```
S() {
  if (c == 'if')
    IFS();
  else
    match('<other>');
}

EL() {
  if (c == 'else') {
    match('else');
    S();
  } else return;
}
```

```
IFS() {
  match('if');
  match('(');
  EX();
  match(')');
  S();
  EL();
}

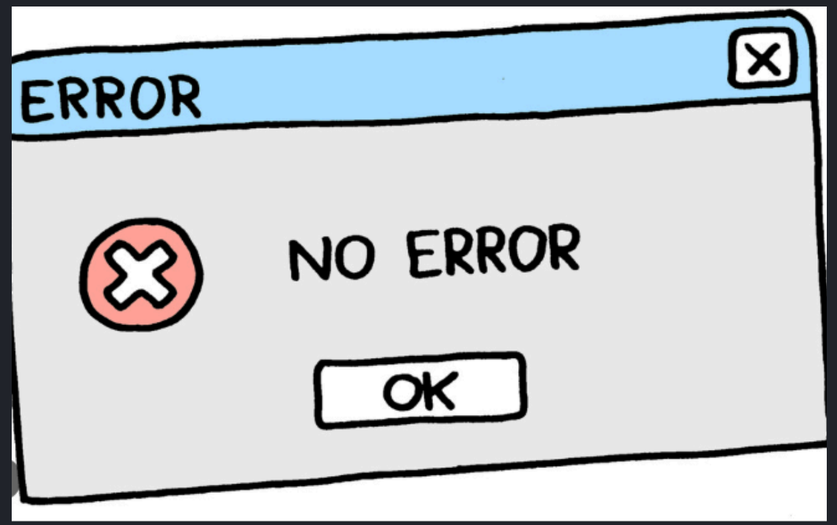
EX() {
  if (c == '0') match('0');
  else match('1');
}
```

Mensajes de Error

Una entrada vacía en la tabla LL(1) delata la hilera inválida. La estrategia mínima reporta el error ahí y no revisa el resto.

Mejor enfoque

- Mensajes de error: dar el mejor diagnóstico posible de la causa.
- Recuperación de error: volver a sincronizar el autómata para reportar otros errores en la hilera.



La estrategia

- Reportar “error de sintaxis aquí”.
- No revisar el resto de la hilera.

	<state>	;	$
SQ	1		
S	4		
SQ'		2	3

Dar buenos mensajes es casi un arte

```
>>> x = (a+b) * c+d)
SyntaxError: invalid syntax
```



- Distintos compiladores dan mensajes diferentes para el mismo error.
- Hay muchas causas posibles; depende de cada lenguaje y situación.
- A veces se hacen de manera incremental:
  - Por un solo número de error (ERROR 17)
  - Ver ejemplos de error e ir mejorando redacción

Ejemplo 1:

1. Consideremos la siguiente gramática:

   1. S → a
   2. S → (S R
   3. R → , S R
   4. R →)

2. Generar "S-expressions", o sea hileras de la forma:

   a
   (a)
   (a,a,a)
   (a,(a,a))
   ((a,(a,a),(a,a)),a)

Tabla:

	a	,	(	)	$
S	1		2		
R		3		4	
$					:)

Las casillas vacías de la tabla son errores de sintaxis.

- Todos los mensajes podrían ser “Aparece ___ cuando se esperaba ___”.
- Calza bien para el match, pero puede confundir entre no terminal y terminal.

1. S → a
2. S → (S R
3. R → , S R
4. R →)

	a	,	(	)	$
S	1	a	2	b	c
R	d	3	e	4	f
$	g	h	i	j	:)

Cada fila revela una causa distinta

Fila	Causa	Ejemplos
S: a, b, c	Se esperaba una S-expression o quedó incompleta	($ (a, $
R: d, e, f	Falta una coma o un cierre; R no dice nada a quien programa	(a a)$ (a,a$
$: g - j	Sobra texto después de una S-expression completa	(a)a$ (a)(a))$

Errores a, b y c: fila S

En a y b se esperaba una S-expression y esos inicios son inválidos:

Aparece ‘,’ cuando se esperaba S-expression

Aparece ‘)’ cuando se esperaba S-expression

En c, el lookahead es el fin de la hilera y la expresión no se completa:

```
( $    (a, $    $
S-expression incompleta
```

Errores d, e y f: fila R

R no tiene un significado evidente para los programadores.
Un mensaje apropiado:

‘a’ aparece cuando se esperaba ‘,’ o ‘)’
‘(’ aparece cuando se esperaba ‘,’ o ‘)’
Fin de hilera aparece cuando se esperaba ‘,’ o ‘)’

O más directo, según la hilera:

Error	Hileras	Mensaje
d, e	(a a)$ ((a)a)$ (a(a))$ ((a)(a))$	Falta ‘,’
f	(a,a$ ((a),a$ (a$	Falta ‘)’

Errores g, h, i y j: fila $

Las hileras que los causan son de la forma:

(a)a$
((a),a),a)$
(a)(a))$

Mensaje de ERROR: ___ aparece después de fin de S-expression

Una tabla de mensajes para cada situación

Error	Mensaje
a	Aparece ‘,’ cuando se esperaba S-expression
b	Aparece ‘)’ cuando se esperaba S-expression
c	S-expression incompleta
d	Falta ‘,’
e	Falta ‘,’
f	Falta ‘)’
g	‘a’ aparece después de fin de S-expression
h	‘,’ aparece después de fin de S-expression
i	‘(’ aparece después de fin de S-expression
j	‘)’ aparece después de fin de S-expression

¡Vayan a la Marcha!



